Algebraic Forms of Boolean Functions

- ***Literal***: A variable, complemented or uncomplemented.
- ***Product term***: A literal or literals ANDed together.
- ***Sum term***: A literal or literals ORed together.

- ***SOP (Sum of Products)***:
- ORing product terms
- $f(A, B, C) = ABC + A'C + B'C$

- ***POS (Product of Sums)***
- ANDing sum terms
- $f(A, B, C) = (A' + B' + C')(A + C')(B + C')$

Algebraic Forms of Boolean Functions

- A ***minterm*** is a product term in which all the variables appear exactly once either complemented or uncomplemented.
- ***Canonical Sum of Products*** (***canonical SOP***):
  - Represented as a sum of minterms only.
  - ***Example***: $f_1(A,B,C) = A'BC' + ABC' + A'BC + ABC$
- Minterms of three variables:

Minterm	Minterm Code	Minterm Number
$A'B'C'$	000	m_0
$A'B'C$	001	m_1
$A'BC'$	010	m_2
$A'BC$	011	m_3
$AB'C'$	100	m_4
$AB'C$	101	m_5
ABC'	110	m_6
ABC	111	m_7

Algebraic Forms of Boolean Functions

- Compact form of canonical SOP form:

  $\boldsymbol{f_1(A,B,C) = m_2 + m_3 + m_6 + m_7}$
- $\boldsymbol{f_1(A,B,C)}$ can also be written in the following form:

  $f_1(A,B,C) = \Sigma\, m\,(2,3,6,7)$ (minterm list form)
- The ***order of variables*** in the functional notation is important.
- Deriving truth table of $f_1(A,B,C)$ from minterm list:

Row No. (i)	**Inputs** ABC	**Outputs** $f_1(A,B,C)= \Sigma m(2,3,6,7)$	**Complement** $f_1'(A,B,C)= \Sigma m(0,1,4,5)$
0	000	0	1 ← m_0
1	001	0	1 ← m_1
2	010	1 ← m_2	0
3	011	1 ← m_3	0
4	100	0	1 ← m_4
5	101	0	1 ← m_5
6	110	1 ← m_6	0
7	111	1 ← m_7	0

Algebraic Forms of Boolean Functions

- ***Example***: Given $f(A,B,Q,Z) = A'B'Q'Z' + A'B'Q'Z + A'BQZ' + A'BQZ$, express $f(A,B,Q,Z)$ and $f'(A,B,Q,Z)$ in minterm list form.

$$
\begin{aligned}
f(A,B,Q,Z) &= A'B'Q'Z' + A'B'Q'Z + A'BQZ' + A'BQZ \\
&= m_0 + m_1 + m_6 + m_7 \\
&= \Sigma\, m(0, 1, 6, 7)
\end{aligned}
$$

$$
\begin{aligned}
f'(A,B,Q,Z) &= m_2 + m_3 + m_4 + m_5 + m_8 + m_9 + m_{10} + m_{11} + m_{12} \\
&\quad + m_{13} + m_{14} + m_{15} \\
&= \Sigma\, m(2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 14, 15)
\end{aligned}
$$

- $AB + (AB)' = AB + A' + B' = 1$, but $AB + A'B' \neq 1$.

Algebraic Forms of Boolean Functions

- A ***maxterm*** is a sum term in which all the variables appear exactly once either complemented or uncomplemented.
- ***Canonical Product of Sums*** (***canonical POS***):
  - Represented as a product of maxterms only.
  - ***Example***: $f_2(A,B,C) = (A+B+C)(A+B+C')(A'+B+C)(A'+B+C')$
- Maxterms of three variables:

Maxterm	Maxterm Code	Maxterm Number
$A+B+C$	000	M_0
$A+B+C'$	001	M_1
$A+B'+C$	010	M_2
$A+B'+C'$	011	M_3
$A'+B+C$	100	M_4
$A'+B+C'$	101	M_5
$A'+B'+C$	110	M_6
$A'+B'+C'$	111	M_7

Algebraic Forms of Boolean **Functions**

- $f_2(A,B,C) = M_0M_1M_4M_5$

  $= \Pi M(0,1,4,5)$
  (maxterm list form)

- The truth table for $f_2(A,B,C)$:

ABC	A+B+C	M	F
000	1+1+1	7	
001	1+1+0	6	
010	1+0+1	5	1
011	1+0+0	4	1
100	0+1+1	3	
101	0+1+0	2	
110	0+0+1	1	1
111	0+0+0	0	1

Algebraic Forms of Boolean Functions

- Truth tables of $f_1(A,B,C)$ and $f_2(A,B,C)$ are identical.
- Hence, $f_1(A,B,C) = \Sigma\, m\,(2,3,6,7)$

  $= f_2(A,B,C)$

  $= \Pi M(0,1,4,5)$
- ***Example: Given*** $f(A,B,C) = (A+B+C')(A+B'+C')(A'+B+C')(A'+B'+C')$, construct the truth table and express it in both maxterm and minterm form.
  - $f(A,B,C) = M_1M_3M_5M_7 = \Pi M(1,3,5,7) = \Sigma\, m\,(0,2,4,6)$

Row No. (i)	**Inputs** ABC	**Outputs** $f(A,B,C) = \Pi\, M(1,3,5,7) = \Sigma m(0,2,4,6)$
0	000	1 m_0
1	001	0 $\leftarrow$ M_1
2	010	1 m_2
3	011	0 $\leftarrow$ M_3
4	100	1 m_4
5	101	0 $\leftarrow$ M_5
6	110	1 m_6
7	111	0 $\leftarrow$ M_7

Algebraic Forms of Boolean Functions

- **Relationship between minterm m_i and maxterm M_i:**
  - For $f(A,B,C)$, $(m_1)' = (A'B'C)' = A + B + C' = M_1$
  - In general, $(m_i)' = M_i$

$$(Mi)' = ((m_i)')' = m_i$$

Karnaugh map (K-map)

•A diagram made up of squares, with each square representing one minterm.

•The squares corresponding to minterms that produce 1 for the function is marked by a 1;

•The others are marked by a 0 or are left empty.

F = AB

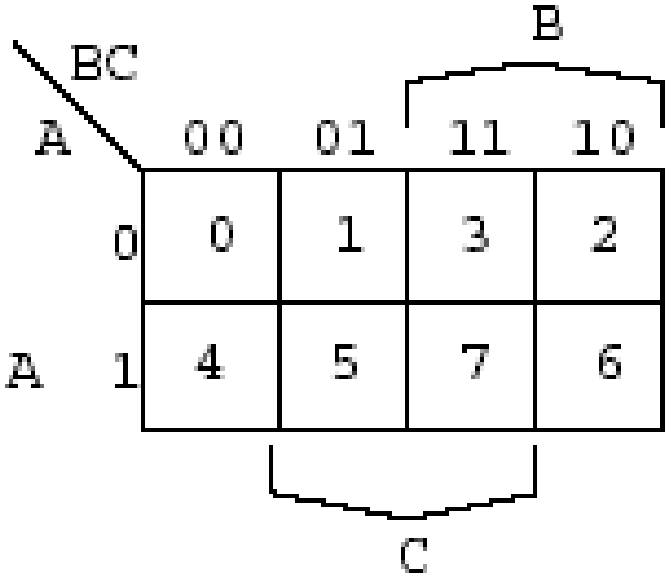
		B'	B
	A \ B	0	1
A'	0	0	1
A	1	2	3

A \ B	0	1
0		
1		1

• If there are more than two variables, we divide them into two groups, and assign the first half to the rows and the other half to the columns.

•The minterm numbers are assigned in an orderly arrangement such that adjacent squares represent minterms that differ by only one variable.

•The squares at the extreme ends of the same are also considered to be adjacent (wraparound condition).



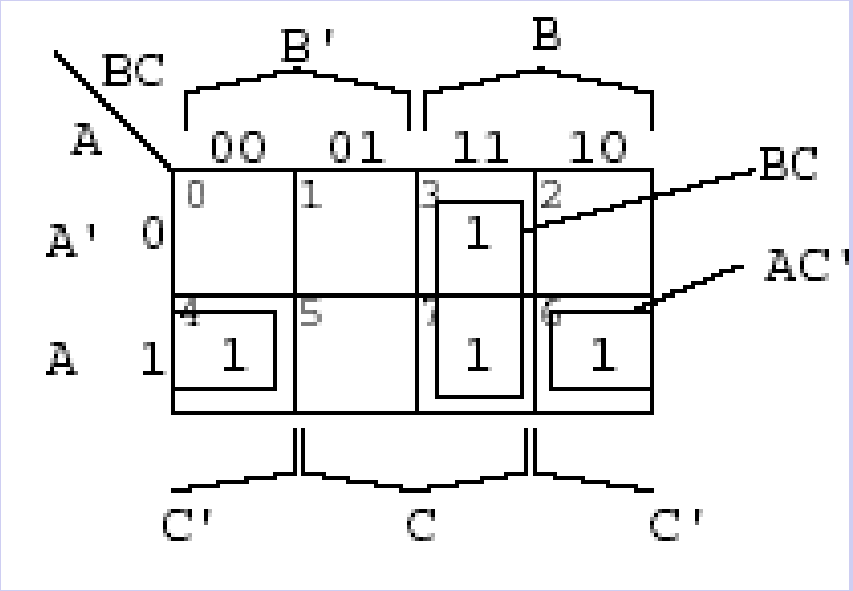
Karnaugh Map (K-map)

• 3-variable, **sum-of-products form**:

(Example)

F(A, B, C) = Σ (3, 4, 6, 7)

= BC +AC′



Sum-of-products procedure

1. Represent the Boolean function in a truth table.

2. Plot the truth table into a Karnaugh map by inserting 1's in those squares where the function is 1.

3. Combine adjacent squares marked by 1 into groups.

4. Represent each group of adjacent squares as an algebraic term using ANDs.

5. A Boolean expression is obtained by combining these terms by OR.

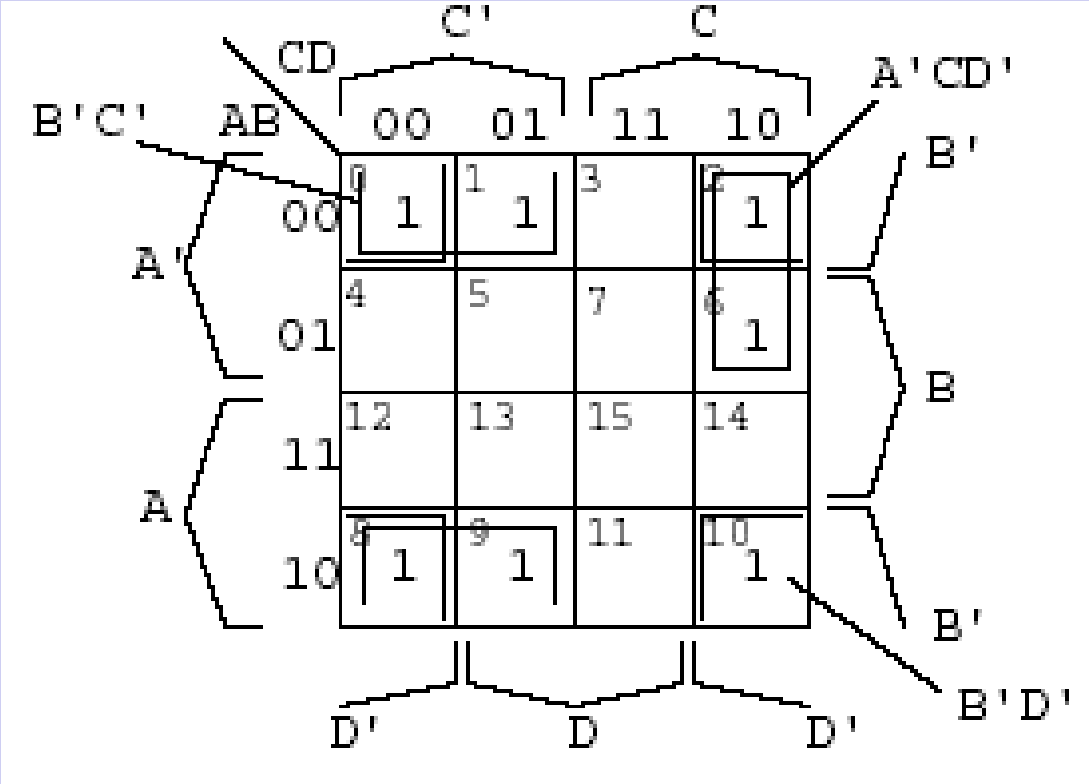
• The terms representing adjacent-square groups may be overlapped when taking ORs of them.

(Example)

Use K-map to show that:

F(A, B, C, D) = Σ(0, 1, 2, 6, 8, 9, 10)

= B′D′ + B′C′ + A′CD′



* Group two columns (e.g., D’) rather than using a single column (e.g., CD’) to save an AND gate.

Minimum SOP and POS

- The *minimum sum of products (MSOP)* of a function, *f*, is a SOP representation of *f* that contains the fewest number of product terms and fewest number of literals of any SOP representation of *f*.
- *f= (xyz +x`yz+ xy`z+ .....*

Is called sum of products.

The + is sum operator which is an OR gate.

The product such as xy is an AND gate for the two inputs x and y.

xyz=1.1.1=1, when x=1,y=1,z=1, while any thing else is zero,

x`yz=1 when x=0,y=1,z=1 any thing else is zero.

Minimization of Boolean expressions

- *The minimization will result in reduction of the number of gates (resulting from less number of terms) and the number of inputs per gate (resulting from less number of variables per term)*
- *The minimization will reduce cost, efficiency and power consumption.*
- **$y(x+x`)=y.1=y$**
- **$y+xx`=y+0=y$**
- **$(x`y+xy`)=x\oplus y$**
- **$(x`y`+xy)=(x\oplus y)`$**
- **$(y+x)(y+x`)=y+y(x+x`)+x.x`=y$**

Example

- ***Minimize the following Boolean function using sum of products (SOP):***
- ***f(a,b,c,d)*** **= ∑*m*(3,7,11,12,13,14,15)**

	abcd	
3	0011	**a`b`cd**
7	0111	**a`bcd**
11	1011	**ab`cd**
12	1100	**abc`d`**
13	1101	**abc`d**
14	1110	**abcd`**
15	1111	**abcd**

- $f(a,b,c,d) = \sum m(3,7,11,12,13,14,15)$
- =a`b`cd + a`bcd + ab`cd + abc`d`+ abc`d + abcd` + abcd
- =cd(a`b`+a`b+ab`+ab)+ab(c`d`+c`d+cd`+cd)
- =cd(a`[b` + b] + ab`+ab) + ab(c`[d` + d] + c[d` + d])
- =cd(a`[1] + ab`+ab) + ab(c`[1] + c[1])
- =a*b* +*cd(*a` + *a)*
- =ab+cd

Minimum product of sums (MPOS)

- The *minimum product of sums (MPOS)* of a function, *f*, is a POS representation of *f* that contains the fewest number of sum terms and the fewest number of literals of any POS representation of *f*.
- The zeros are considered exactly the same as ones in the case of sum of product (SOP)

Example: ***Minimum product of sums (MPOS)***

$f(a,b,c,d) = \prod M(0,1,2,4,5,6,8,9,10)$

$=\sum m(3,7,11,12,13,14,15)$

$=[(a+b+c+d)(a+b+c+d`)(a+b+c`+d)$

$(a+b`+c+d)(a+b`+c+d`)(a+b`+c`+d)$

$(a`+b+c+d)(a`+b+c+d`)(a`+b+c`+d)]$

Group each two terms that have at least one complementary pair and multiply them,

$(a+b+c+d)(a+b+c+d`)=a+b+c$

we can repeat terms that have been multiplied also.

$=(a+b+c+d)(a+b+c`+d)=a+b+d$

$f=[(\underline{a+b}+c+\underline{d})(a+b+c+d`)(\underline{a+b}+c`+\underline{d})$

$(\underline{a+b`}+c+\underline{d})(a+b`+c+d`)(\underline{a+b`}+c`+\underline{d})$

$(\underline{a`+b}+c+\underline{d})(a`+b+c+d`)(\underline{a`+b}+c`+\underline{d})]$

$f=[(a+b+c)(\underline{a+b}+\underline{d})(\underline{a+b`}+c)(\underline{a+b`}+\underline{d})\ (\underline{a`+b}+c)(\underline{a`+b}+\underline{d})]$

$f=[(\underline{a}+b+\underline{c})(\underline{a}+b+\underline{d})(\underline{a}+b`+\underline{c})(\underline{a}+b`+\underline{d})\ (a`+b+c)(a`+b+d)]$

$f=[(\underline{a}+\underline{c})(\underline{a}+\underline{d})(b+c)(b+d)]= [(a+ad+ac+cd)(b+c)(b+d)$

$= (a(1+d+c)+cd)(b+c)(b+d)=(a+cd)(b+cd) =(ab+cd)$

0	a+b+c+d
1	a+b+c+d`
2	a+b+c`+d
4	a+b`+c+d
5	a+b`+c+d`
6	a+b`+c`+d
8	a`+b+c+d
9	a`+b+c+d`
10	a`+b+c`+d

Karnaugh Maps (K-maps)

- Karnaugh maps -- A tool for representing Boolean functions of up to six variables.
- K-maps are tables of rows and columns with entries represent 1`s or 0`s of SOP and POS representations.
- An *n*-variable K-map has 2^n cells with each cell corresponding to an *n*-variable truth table value.
- K-map cells are labeled with the corresponding truth-table row.
- K-map cells are arranged such that adjacent cells correspond to truth rows that differ in only one bit position (*logical adjacency*).
- The gray code is suitable for representing the rows or columns of K-maps
- Boolean functions are mapped (or plotted) by placing the function`s value (*0,1,d*) in each cell

3	0011	**a`b`cd**
7	0111	**a`bcd**
11	1011	**ab`cd**
12	1100	**abc`d`**
13	1101	**abc`d**
14	1110	**abcd`**
15	1111	**abcd**

Plotting (Mapping) Functions in Canonical Form on a K-map

- Let f be a Boolean function of n variables where $n \leq 6$.
- Assume that the cells of the K-map are numbered from 0 to 2^n where the numbers correspond to the rows of the truth table of f *and ordered according to a gray code*.
- If m_i is a minterm of f, then place a 1 in cell i of the K-map.
- If M_i is a maxterm of f, then place a 0 in cell i.
- If d_i is a don't care of f, then place an x in cell i.

Examples

- *Two variable K-map*
- $f(A,B)=\sum m(0,1,3)=A`B`+A`B+AB$

- Use the product of sums (Max terms) to write *f*

B \ A	0	1
0	1	0
1	1	1

K-Map
Three variable map

- $f(A,B,C) = \sum m(0,3,5) =$ A`B`C`+A`BC+AB`C

- Notice that the K-map is cyclic
- This means all following produce the same K-map;
- 00-01-11-10
- 01-11-10-00
- 11-10-00-01
- 10-00-01-11
- Any code pair differs

in one bit position only.

	A`B`	A`B	AB	AB`
C`	1 A`B`C`			
C		1 A`BC		1 AB`C

7:48 AM

Maxterm example

$f(A,B,C) = \prod M(1,2,4,6,7)$

=(A+B+C`)(A+B`+C)(A`+B+C) )(A`+B`+C) )(A`+B`+C`)

Note that the complements are (0,3,5) which are the minterms of the previous example

		(A+B)	(A+B`)	(A`+B`)	(A`+B)
		A`B`	A`B	AB	AB`
C	C`		0	0	0
C`	C	0		0	

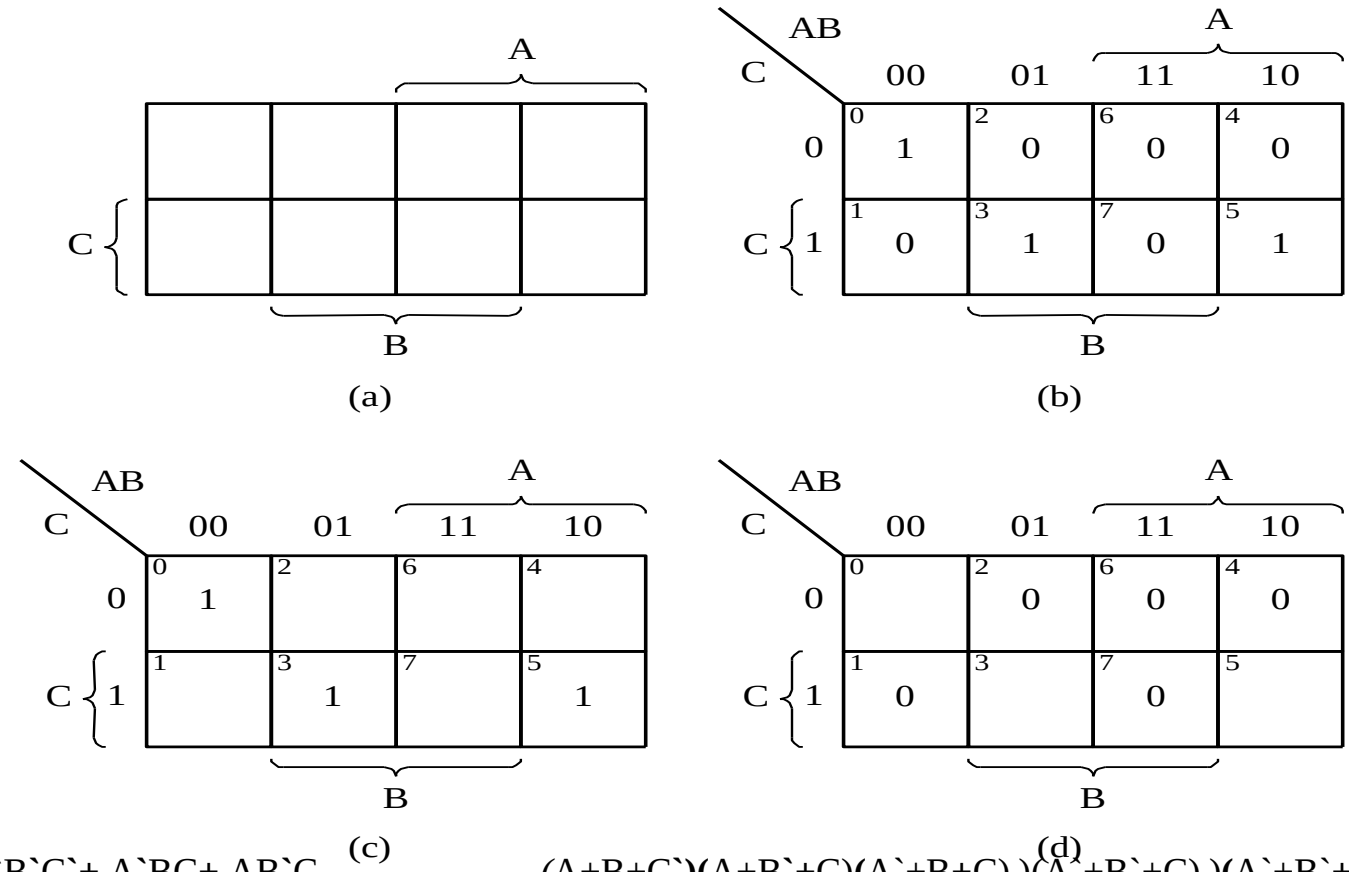
Plotting functions on K-maps

$f(A,B,C) = \sum m(0,3,5) = \prod M(1,2,4,6,7)$

Ones of $f(A,B,C)$ = A`B`C`+ A`BC+ AB`C

Or zeros of

$f(A,B,C)$ = (A+B+C`)(A+B`+C)(A`+B+C) )(A`+B`+C) )(A`+B`+C`)



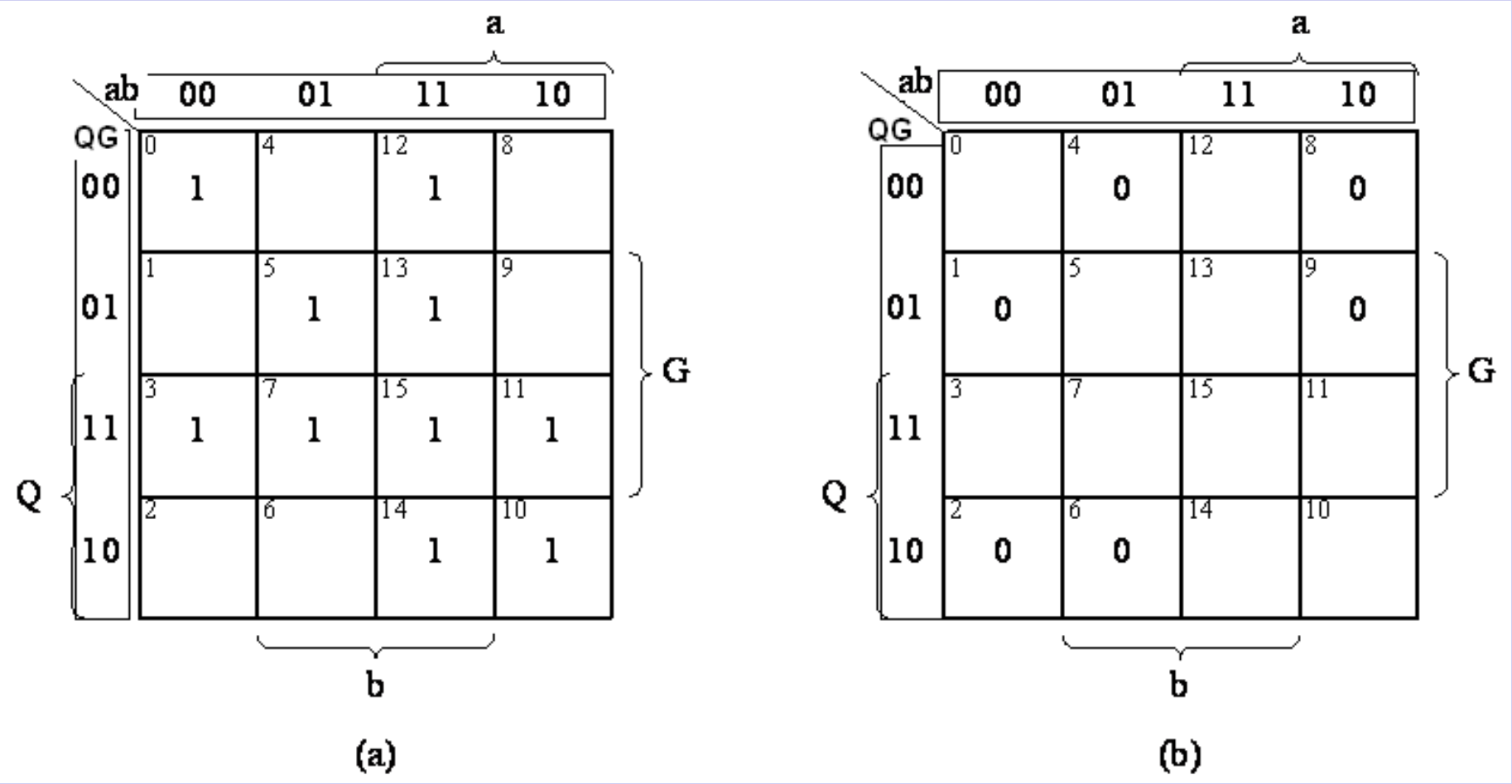
A`B`C`+ A`BC+ AB`C

(A+B+C`)(A+B`+C)(A`+B+C) )(A`+B`+C) )(A`+B`+C`)

Four variable k-map example

(a) Minterm form. (b) Maxterm form.

$f(a,b,Q,G) = \sum m(0,3,5,7,10,11,12,13,14,15) = \prod M(1,2,4,6,8,9)$



$f(a,b,Q,G) = a`b`G`Q`+ab+Gb+QG+aQ$

Simplification of Boolean Functions Using K-maps

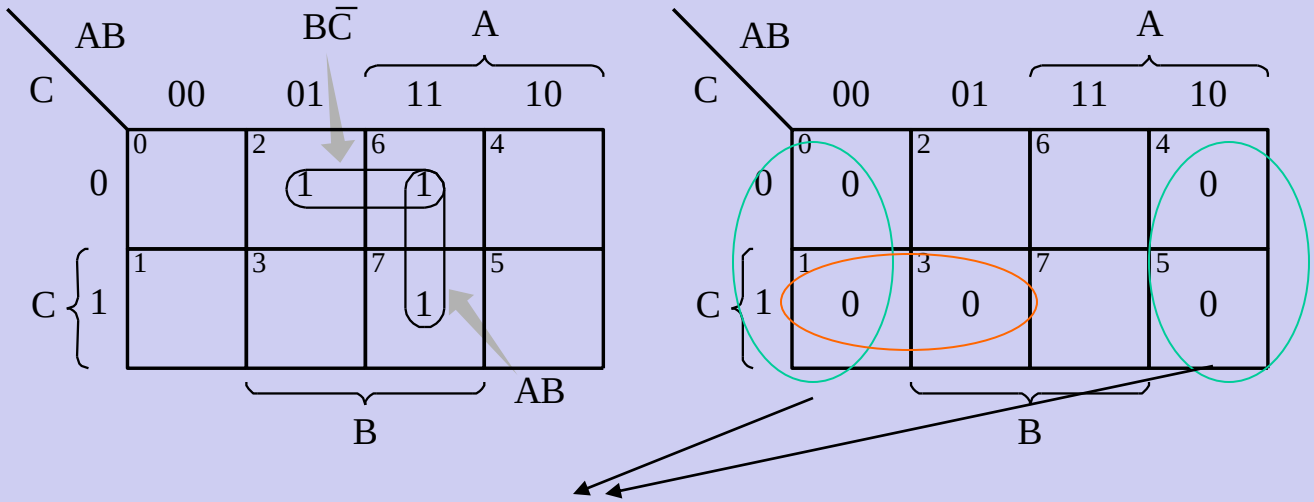
- K-map cells that are physically adjacent are also logically adjacent.
- Cells on an edge of a K-map are logically adjacent to cells on the opposite edge of the map.
- If two logically adjacent cells both contain logical value 1, the two cells can be combined.
- The two cells are combined to eliminate any variable title that changes (from complement to un-complement or vice-versa)
- This is equivalent to the algebraic operation, $a\mathrm{P} + a'\mathrm{P} = \mathrm{P}$ where P is a product term not containing a or a'.
- Cells may be combined in groups of $2,4,8,\ldots,2^k$.
- Always begin with the largest group, which means if you can find one 8 members group is better than 2 or 4 groups.

Example

Simplify f= A`BC`+ A B C`+ A B C **using;**

(a) Sum of minterms. (b) Maxterms.

- Each cell of an *n*-variable K-map has *n* logically adjacent cells.



F`= B`+ A`C  F = B(A+C`)

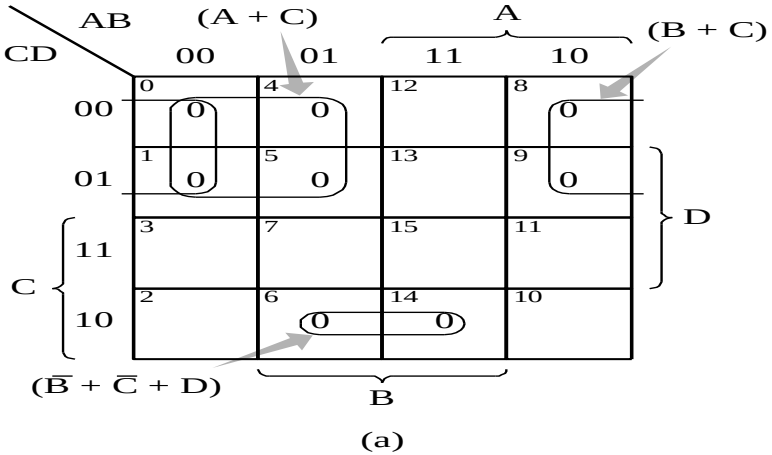
a- $f(A,B,C) = AB + BC'$ *b-* $f(A,B,C) = B(A + C')$

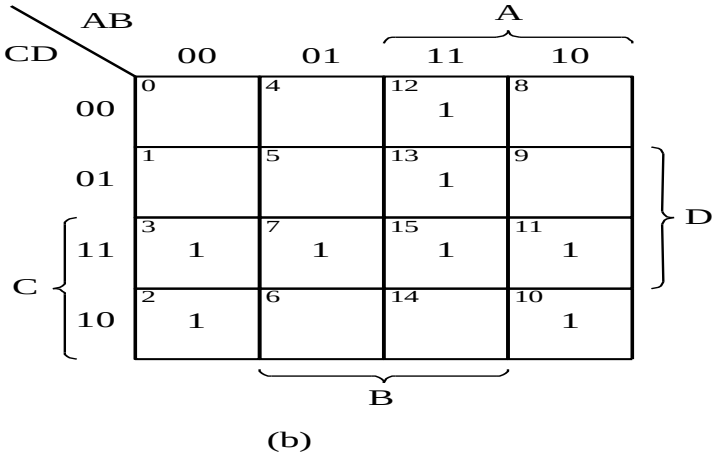
Example f=ABC′D′+ ABC′D+ ABCD+ A′B′CD

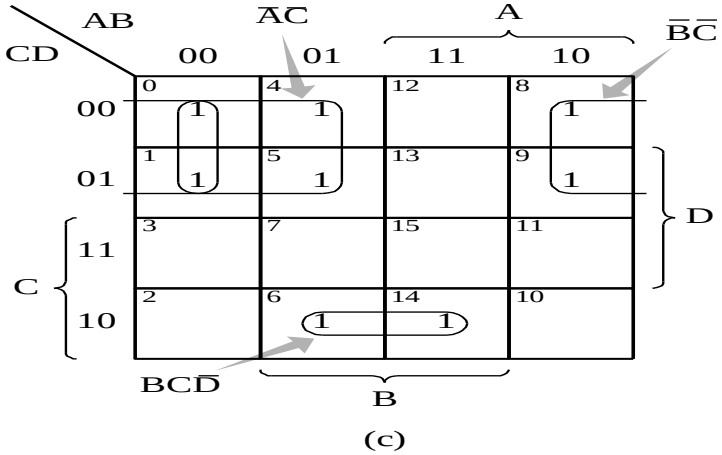
$+ A'B'CD' + A'BCD + AB'CD + AB'CD'$

(a) Maxterms, (b) Minterms, (c) Minterms of f'.

$$f(A,B,C,D) = (A + C)(B + C)(B' + C' + D)$$







7:48 AM

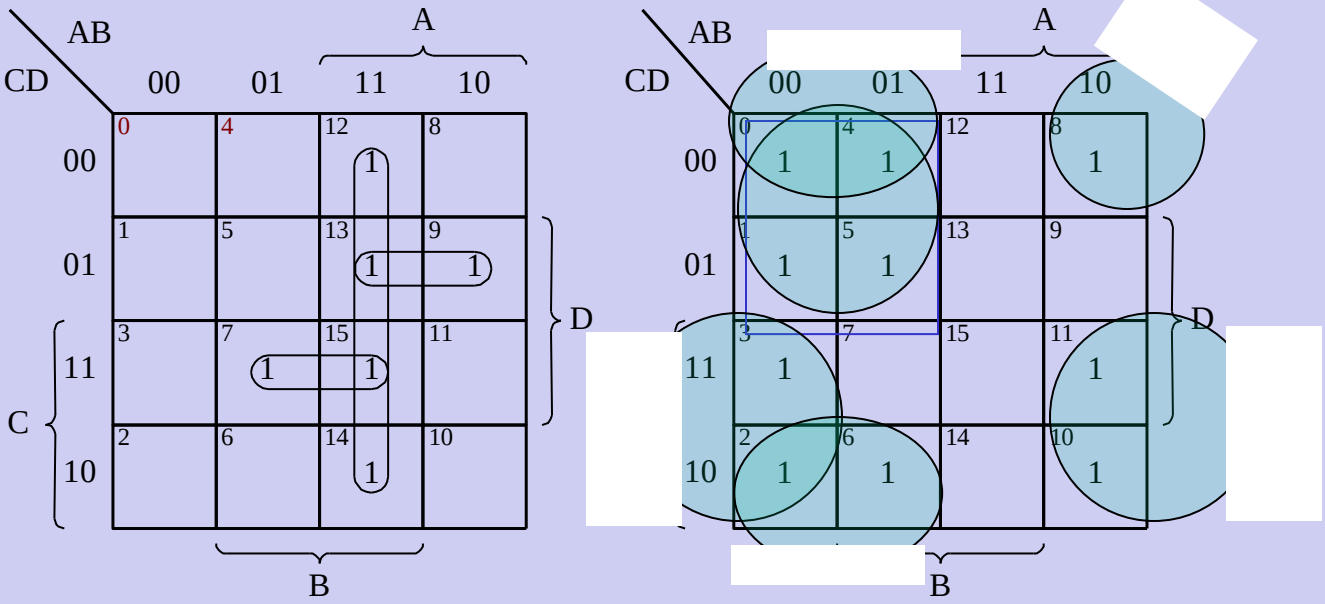
Example: simplify f=∑(0,4,8,1,5,3,11,2,6,10) using;

(a) K-map of f', (b) K-map of f.

Note that both functions are treated as minterms

$$f'(A,B,C,D)= (AB)+(AC'D)+(BCD)$$

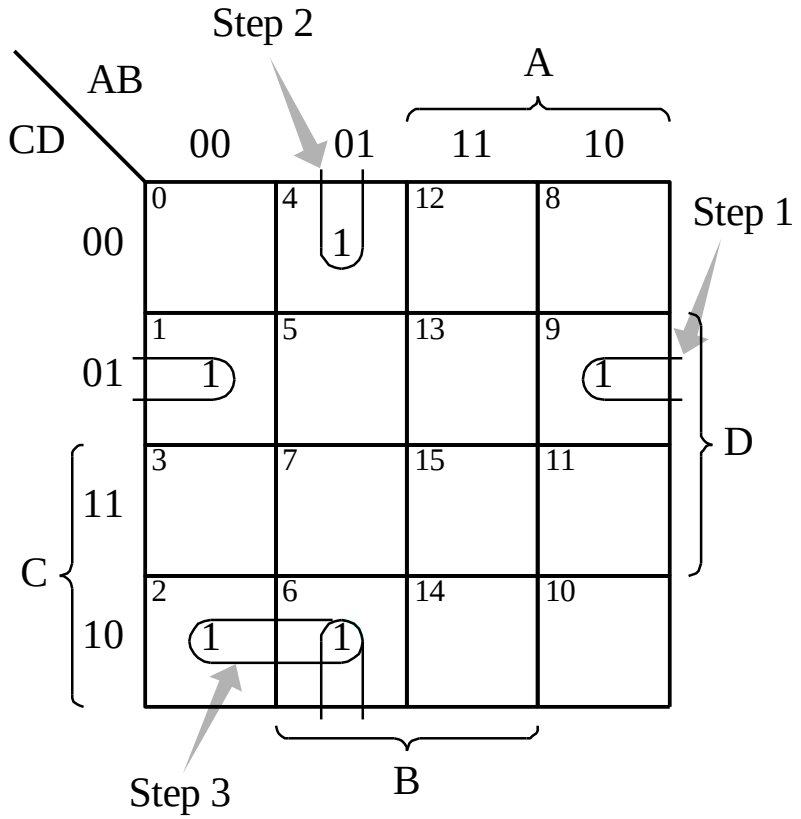
$$f(A,B,C,D)= (A'+B')(A'+C+D')(B'+C'+D')$$



$$b\text{-}\ f(A,B,C,D)= (A'C')+(A'D')+(B'C)+(B'D')$$

Example

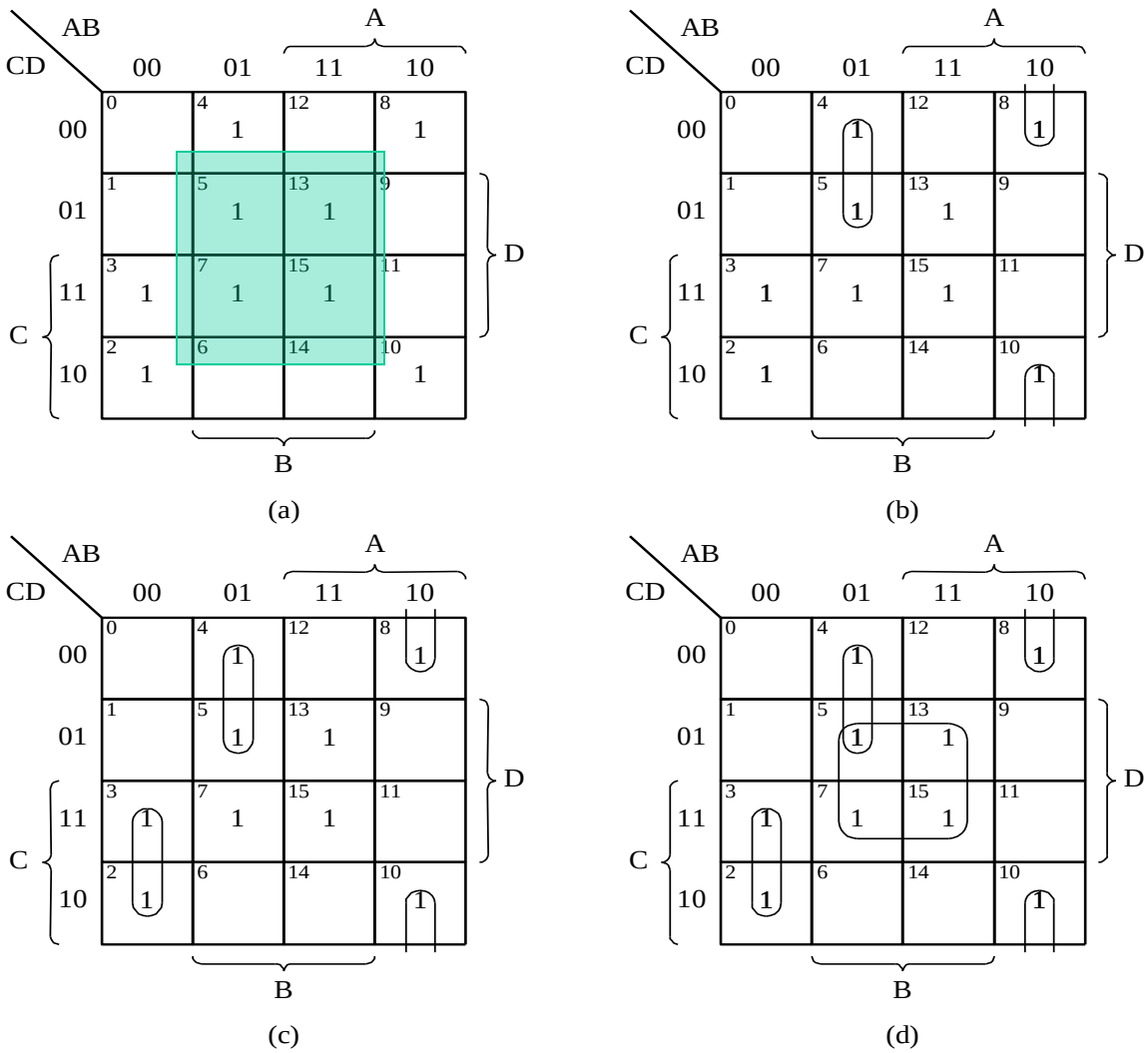
$$f(A,B,C,D) = \sum m(1,2,4,6,9)$$



$$f(A,B,C,D) = B'C'D + A'BD' + A'CD'$$

Example Simplify

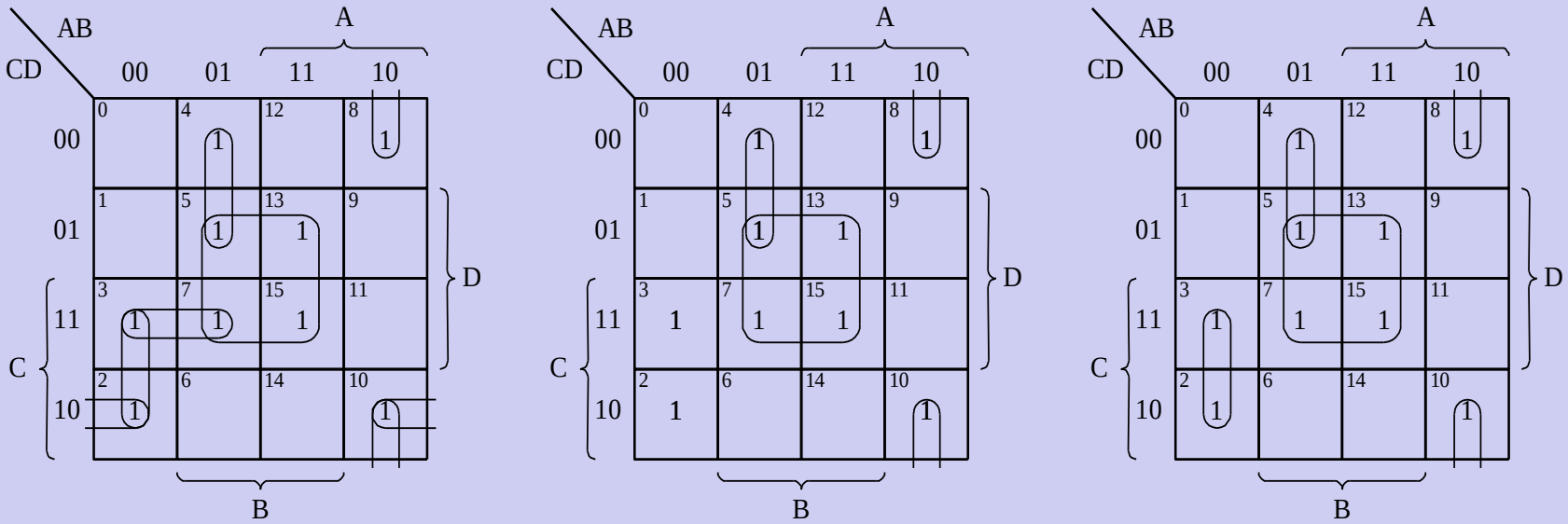
$$f(A,B,C,D) = \sum m(2,3,4,5,7,8,10,13,15)$$



7:48 AM

Example Multiple selections

$$f(A,B,C,D) = \sum m(2,3,4,5,7,8,10,13,15)$$



(a) (b) (c)

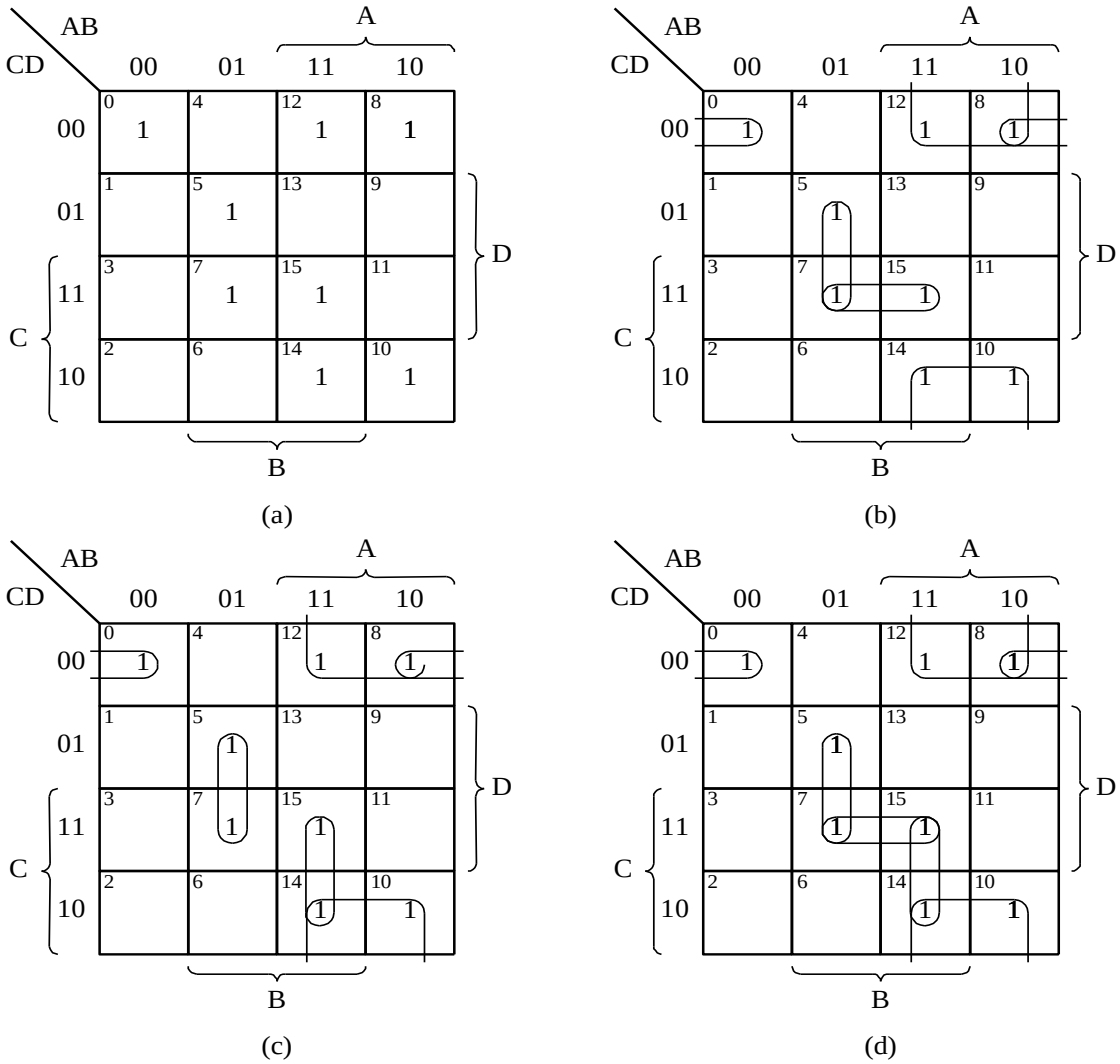
c produces less terms than a, (remove redundant groups from a to get c.
Remove the groups that contain A` and B`

7:48 AM

$$f(A,B,C,D) = BD + A'BC' + AB'D' + A'B'C$$

Example Redundant selections

$$f(A,B,C,D) = \sum m(0,5,7,8,10,12,14,15)$$

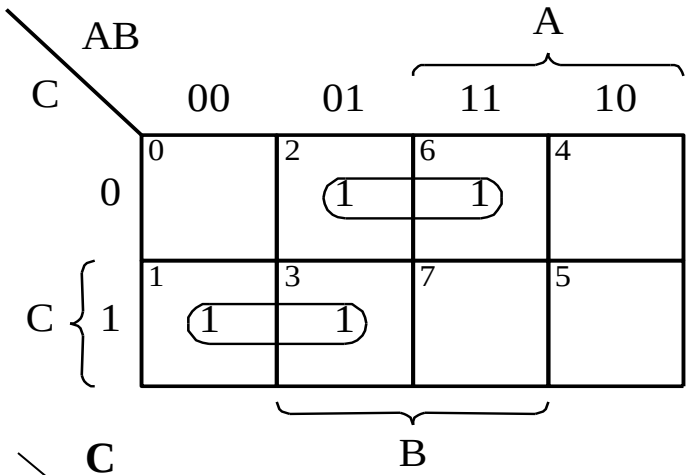


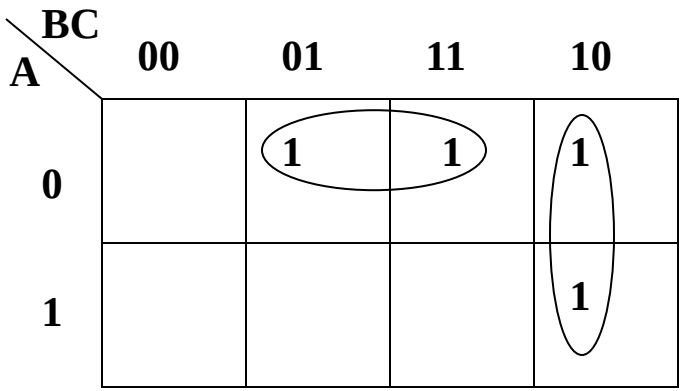
7:48 AM

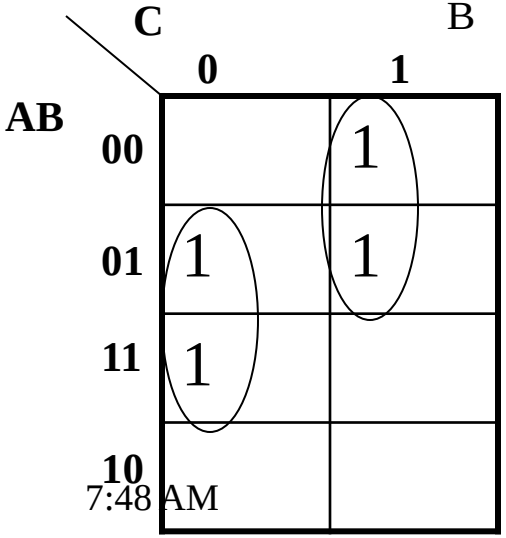
Different styles of drawing maps

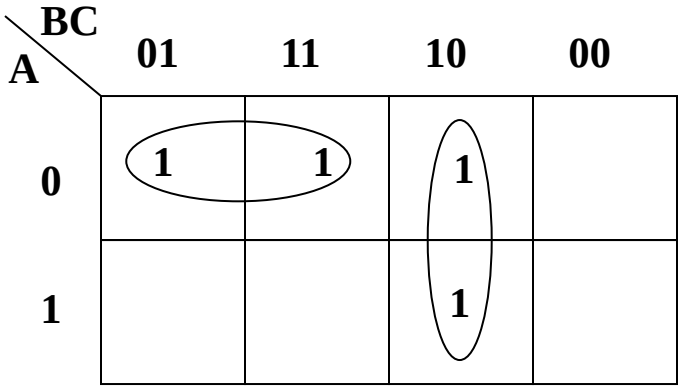
$f(A,B,C) = \sum m(1,2,3,6) = A'C + BC'$

all the following map are equivalent









Don’t-care condition

•Minterms that may produce either 0 or 1 for the function.

•They are marked with an × in the K-map.

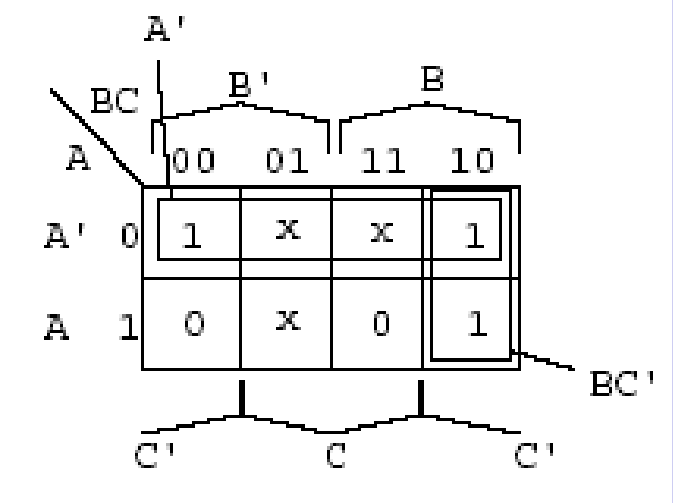
•This happens, for example, when we don’t input certain minterms to the Boolean function.

•These don’t-care conditions can be used to provide further simplification of the algebraic expression.

(Example) F = A`B`C`+A`BC` + ABC`

d=A`B`C +A`BC + AB`C

F = A` + BC`



Five variable K-maps

a`	f
0	1
1	
2	
3	
4	
5	1
6	
7	1
8	
9	
10	
11	
12	
13	1
14	
15	1

We can use 2 four variable K-map to map a five variable function:

$$f(a,b,c,d,e) = \sum m(0,5,7,13,15,16,21,23,29,31)$$

Divide the minterms into two groups according to the most significant bit

a	f
16	1
17	
18	
19	
20	
21	1
22	
23	1
24	
25	
26	
27	
28	
29	1
30	
31	1

a`=0 map

de \ bc	00	01	11	10
00	0 1			
01		5 1	13 1	
11		7 1	15 1	
10				

a=1 map

de \ bc	00	01	11	10
00	16 1			
01		21 1	29 1	
11		23 1	31 1	
10				

Minimize each map and Or the two results to get the minimization for the variable function.

a`=0 map

de \ bc	00	01	11	10
00	1			
01		1	1	
11		1	1	
10				

a=1 map

de \ bc	00	01	11	10
00	1			
01		1	1	
11		1	1	
10				

$F1=a`b`c`d`e` + a`ce,$

$F2=ace + ab`c`d`e`$

$f(a,b,c,d,e) =$ f1+f2

$$F=(a+a`)ce + (a+a`)b`c`d`e`$$
$$=ce + b`c`d`e`$$

Six variable K-maps

We can use 4 four variable K-map to map a six variable function:

$f(a,b,c,d,e,f) = \sum m(0,5,7,13,15,16,21,23,29,31,32,48)$

Divide the minterms into 4 groups using the 2 most significant bits

f1 =a`b`*c`d`e`f`*+{a`b`*c`de`f*
a`b`*cde`f*+a`b`*c`def*
+a`b`*cdef*}=*a`b`[c`d`e`f`+df]*
f2 =a`b*c`d`e`f``*+a`b*c`de`f*
a`b*cde`f*+a`b*c`def*
+a`b*cdef=a`b[c`d`e`f`+df]*
f3 =ab`*c`d`e`f`*
f4 =ab*c`d`e`f`*
F(a,b,c,d,e,f)=f1+f2+f3+f4
=c`d`e`f`(a`b`+a`b+ab`+ab)+
df(a`b`+a`b)
=c`d`e`f`+a`df

a`b`=00 map

ef \ cd	00	01	11	10
00	0 1	4	12	8
01	1	5 1	13 1	9
11	3	7 1	15 1	11
10	2	6	14	10

a`b=01 map

ef \ cd	00	01	11	10
00	16 1	20	28	24
01	17	21 1	29 1	25
11	19	23 1	31 1	27
10	18	22	30	26

ab`=10 map

ef \ cd	00	01	11	10
00	32 1	36	44	40
01	33	37	45	41
11	35	39	47	43
10	34	38	46	42

ab=11 map

ef \ cd	00	01	11	10
00	48 1	52	60	56
01	49	53	61	57
11	51	55	63	59
10	50	54	62	58

7:48 AM

NAND-ONLY LOGIC CIRCUITS

• Any logic circuits can be transformed to an implementation where only NAND gates (and inverters) are used.

• The general approach to finding a NAND-gate realization: Use DeMorgan's theorem to eliminate all the OR operations.

(Example)

F = A + B • (C + D`)

= A + B • (C`D)`

Note that (C`D)` = C + D` and (A`X`)` = A + X

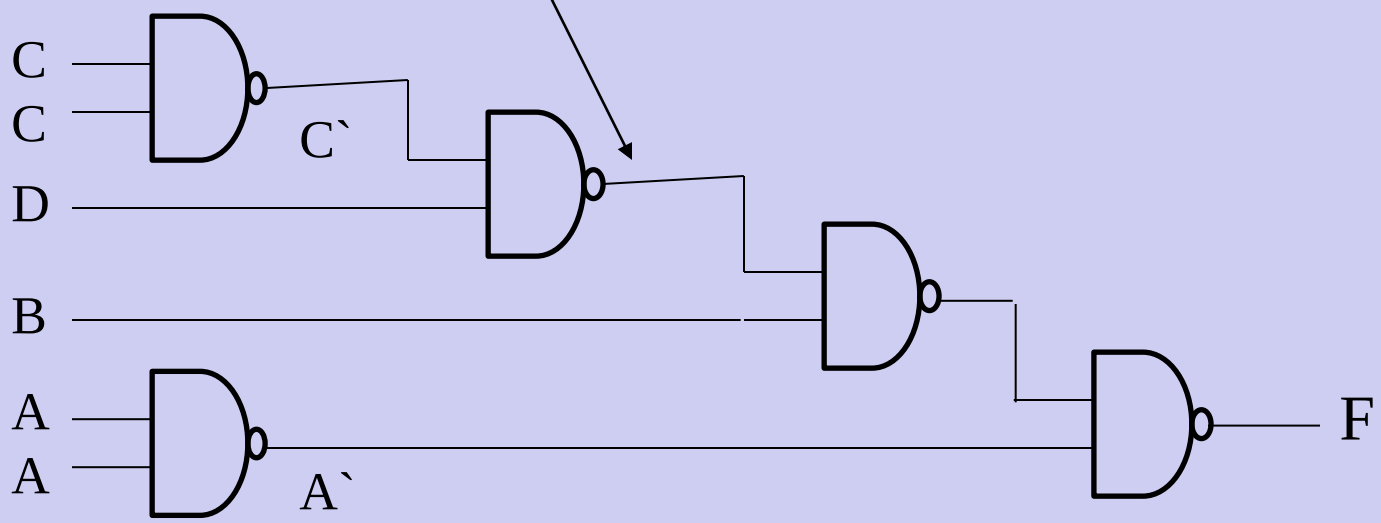
F = (A`• (B • (C`D)`)`)`

Now there is no OR operation in the Boolean expression. Note that

A NAND B = (AB)`

$F = (A` \bullet (B \bullet \boxed{(C`D)`})`)`$

The logic NAND only circuit for this function is given by:



We can also use the same procedure to do NOR only gates.

$F = A + B \bullet (C + D`) = A + [B` + (C + D`)`]`$

$= \{A + [B` + (C + D`)`]`\}``$